SMART DOCTOR FINDER AND APPOINTMENT BOOKING SYSTEM WITH INTELLIGENT FEATURES ON MICROSOFT AZURE

CLOUD COMPUTING

Submitted by

HEMASHRI U (230701114)
ISHWARI RAJMOHAN (230701118)
JAYABHARATHI M (230701126)

in partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

RAJALAKSHMI ENGINEERING COLLEGE

MAY 2025

ANNA UNIVERSITY, CHENNAI

BONAFIDE CERTIFICATE

Certified that this project report titled "**SMART DOCTOR FINDER AND APPOINTMENT BOOKING SYSTEM WITH INTELLIGENT FEATURES ON MICROSOFT AZURE**" is the bonafide work of **HEMASHRI U (230701114), ISHWARI RAJMOHAN (230701118), JAYABHARATHI M (230701126)** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. E. M Malathy
Professor &
Head of The Department
Department of Computer Science
and Engineering
Rajalakshmi Engineering College

SIGNATURE

S. Ponmani
Supervisor
Associate Professor
Department of Computer Science
and Engineering
Rajalakshmi Engineering College

Submitted to Project Viva Voce Examination held on_____

Internal Examiner **External Examiner**

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman **Mr. S.MEGANATHAN, B.E, F.I.E.**, our Vice Chairman **Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S.**, and our respected Chairperson **Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D.,** for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to **Dr. S.N. MURUGESAN, M.E., Ph.D.**, our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to **Dr.MALATHY E.M, Ph.D.**, Professor and Head of the Department of Computer Science and Engineering for her guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our internal guide, **DR. S. PONMANI M.E.,Ph.D.,** Associate Professor, Department of Computer Science and Engineering for his valuable guidance throughout the course of the project.

HEMASHRI U
ISHWARI RAJMOHAN
JAYABHARATHI M

ABSTRACT

Access to healthcare services remains a significant challenge for a large population, particularly in terms of locating nearby doctors, verifying their availability, and securing timely appointments. Patients frequently spend considerable time and effort navigating multiple platforms, only to encounter outdated schedules or unavailable practitioners. Existing general-purpose appointment systems lack intelligent decision-support features such as symptom-based recommendations, real-time load estimation, and automatic fallback to alternative doctors.

This project presents the Smart Doctor Finder and Appointment Booking System, a cloud-based healthcare application deployed on Microsoft Azure, designed to resolve these inefficiencies. The system enables users to search for nearby doctors using location-based services, check live availability, and confirm bookings through a streamlined interface. Six core intelligent features are integrated: location-based doctor search powered by Azure Maps, a symptom-to-specialization recommendation engine, an emergency priority booking workflow, a doctor load prediction module that classifies slots as low, medium, or high demand, an auto doctor switch mechanism that seamlessly redirects users when a selected doctor is at capacity, and cloud-based prescription storage via Azure Blob Storage.

The backend is built using FastAPI (Python) and deployed on Azure App Service, with data persistence provided by Azure Cosmos DB. The frontend is developed using React, delivering a responsive and intuitive user experience. Azure Functions handle background tasks such as appointment reminders and notifications, while Azure Maps API supports geolocation and routing features.

The system architecture adheres to a cloud-native, RESTful design, ensuring scalability, high availability, and security. Intelligent features are implemented through rule-based logic and data aggregation, eliminating the need for complex machine learning pipelines while remaining highly effective. Preliminary evaluation demonstrates sub-300ms API response times, reliable booking workflows, and successful auto-switching under simulated load conditions. The system is expected to meaningfully reduce patient wait times and improve access to appropriate healthcare services.

TABLE OF CONTENTS

CHAPTER 1 - INTRODUCTION

1.1 PROBLEM STATEMENT

Healthcare access remains one of the most critical unresolved challenges in both urban and semi-urban environments. Despite a growing number of registered medical practitioners, patients continue to face significant barriers when attempting to locate, evaluate, and book appointments with doctors who are both nearby and available. The fragmented nature of existing solutions — ranging from manual phone-based bookings to generic online directories — leads to wasted time, missed appointments, and inadequate care.

- **Lack of Real-Time Doctor Availability:** Existing platforms rarely reflect a doctor's live schedule. Patients often arrive at clinics only to learn that the doctor is unavailable or fully booked. Approximately 60% of appointment-related complaints in outpatient settings involve scheduling mismatches that could be prevented with a real-time booking system.
- **Absence of Intelligent Guidance:** Most healthcare search tools require patients to already know their required specialization. This is a significant gap: a patient experiencing chest pain may not know whether to consult a cardiologist or a general physician. Without symptom-based guidance, patients risk consulting the wrong specialist, leading to delayed diagnosis and unnecessary expenditure.
- **Emergency Situations Lack Priority Workflows:** Standard appointment systems are designed for routine bookings and provide no pathway for patients in urgent or emergency situations to quickly reach the nearest available doctor. This gap can have critical consequences in time-sensitive medical scenarios.
- **Inefficient Doctor Load Distribution:** Without visibility into how busy a particular doctor or clinic is, patients and the broader healthcare system cannot effectively distribute load. Certain doctors may be overwhelmed while equally qualified practitioners nearby remain underutilized, increasing wait times across the board.
- **No Automated Fallback for Unavailable Doctors:** When a selected doctor is fully booked, patients must restart the entire search process manually. There is no intelligent mechanism to seamlessly suggest an alternative doctor with the same specialization in the vicinity.
- **Fragmented Health Records:** Patients maintain prescriptions and medical records in physical form, making them prone to loss and difficult to share with new doctors. The absence of a digital prescription locker forces patients to carry physical documents and risk losing critical medical history.

These compounding inefficiencies result in poor patient outcomes, increased healthcare costs, and an overall erosion of trust in digital health systems. A unified, intelligent, cloud-based platform is needed to bridge this gap.

1.2 OBJECTIVE OF THE PROJECT

The primary objective of this project is to design and deploy a cloud-native healthcare appointment system that simplifies the process of finding, evaluating, and booking doctors while integrating intelligent decision-support features to improve patient outcomes and system efficiency.

- **Enable Location-Based Doctor Discovery:** Allow users to instantly find nearby doctors filtered by specialization, using real-time geolocation powered by Azure Maps.
- **Streamline the Appointment Booking Process:** Provide a clean, multi-step booking interface that allows users to select a doctor, choose available time slots, and confirm appointments — all stored securely in Azure Cosmos DB.
- **Implement Symptom-Based Recommendation:** Develop a rule-based recommendation engine that maps user-reported symptoms to appropriate medical specializations, guiding patients toward the right practitioner without requiring medical knowledge.
- **Support Emergency Priority Booking:** Integrate an emergency booking pathway that automatically identifies the nearest available doctor and confirms an appointment with minimal user interaction.
- **Predict and Communicate Doctor Load:** Analyse booking density per doctor per time slot and surface low, medium, or high demand indicators to help users select optimal appointment windows.
- **Automate Doctor Switching:** When a selected doctor reaches capacity, automatically identify and suggest the nearest alternative with the same specialization to ensure continuity of care.

Provide Secure Prescription Storage: Allow users to upload, store, and retrieve medical prescriptions securely via Azure Blob Storage, creating a persistent and accessible digital health record.

1.3 SCOPE AND BOUNDARIES

The scope of this project encompasses the end-to-end design, development, and cloud deployment of a full-stack web application with integrated intelligent features and Azure cloud infrastructure. The system covers user registration and authentication, doctor profile management, location-based search, appointment booking, symptom-based recommendation, emergency booking, load prediction, and prescription file management.

The backend is built using FastAPI (Python) and deployed on Azure App Service. Data persistence is handled by Azure Cosmos DB. Geolocation and mapping services are provided by Azure Maps. File storage for prescriptions uses Azure Blob Storage. Notifications are implemented through Azure Functions. The frontend is developed using React and communicates with all backend APIs via RESTful endpoints.

The system is scoped for individual patient use within a defined geographic radius. Multi-hospital enterprise integration, telemedicine, insurance processing, and payment gateway features are outside the current scope and are deferred to future enhancement phases.

1.4 STAKEHOLDERS AND END USERS

1. **Patients / General Users:** The primary end users who search for doctors, book appointments, manage prescriptions, and use the emergency booking feature.
2. **Doctors / Medical Practitioners:** Registered practitioners whose profiles, specializations, locations, and availability are managed within the system.
3. **Project Developers / Team Members:** Responsible for designing, building, testing, and maintaining all system modules including APIs, frontend, database integration, and cloud deployment.
4. **College / University Evaluators:** Academic supervisors and examiners who assess the technical implementation, cloud deployment, and intelligent features.

1.5 TECHNOLOGIES USED

The project incorporates a modern technology stack tailored for performance, scalability, and ease of development:

Table 1. Frontend Technologies

React	Building the user interface of the application
HTML / CSS / JavaScript	Core web technologies for structure and styling
Axios	HTTP client for communicating with backend APIs

Table 2: Backend Technologies

FastAPI (Python)	Primary backend framework for building RESTful APIs
Uvicorn	ASGI server for running the FastAPI application
Pydantic	Data validation and schema enforcement
Python 3.10	Core programming language for all backend logic

Table 3: Cloud and Infrastructure

Microsoft Azure	Primary cloud provider for all hosting and services
Azure App Service	Deploy and host the FastAPI backend
Azure Cosmos DB	NoSQL database for users, doctors, and bookings
Azure Blob Storage	Secure storage for prescription files

Azure Maps	Location-based doctor search and routing
Azure Functions	Serverless execution for notifications and reminders
Docker	For containerizing applications.

Table 4: AI and Machine Learning

Google Gemini Pro Vision API	For generating descriptions.
Azure Content Safety API	For detecting inappropriate content.

Table 5: DevOps and CI/CD

GitHub Actions	For automating workflows, builds, and deployments.
Docker Hub	As a registry for storing and distributing container images.
Azure CLI	For command-line management of Azure resources.

1.6 ORGANIZATION OF THE REPORT

This report is structured to provide a complete and logical documentation of the entire project lifecycle.

Chapter 1 introduces the problem context, project objectives, scope, stakeholders, technologies, and report structure.

Chapter 2 presents a detailed examination of the system design and architecture, including requirements, solution overview, cloud strategy, infrastructure specifications, and service mappings.

Chapter 3 delves into the DevOps implementation, covering CI/CD pipelines, Terraform automation, Docker containerization, orchestration principles, and AI service integration. **Chapter 4** focuses cloud operations and security, discussing Cosmos DB integration, Blob Storage, Azure Maps usage, and notification workflows..

Chapter 5 presents the results of the implementation, including summaries, challenges overcome, performance metrics, cost analysis, and key learnings.

Chapter 6 concludes the report with a summary of achievements and a roadmap for future enhancements.

The document concludes with references and appendices containing configuration details and system screenshots.

CHAPTER 2 - SYSTEM DESIGN AND ARCHITECTURE

2.1 REQUIREMENT SUMMARY

Functional Requirements

The system consists of multiple cloud-based components deployed primarily on Microsoft Azure to support a scalable, secure, and efficient platform.

The The frontend of the application is developed using React and communicates with a FastAPI backend hosted on Azure App Service. This setup enables a responsive, real-time interface with dynamic data fetching for doctor availability and booking management.

The backend orchestrates all business logic through RESTful API endpoints, handling user authentication, doctor search, appointment booking, and the six intelligent features. Data persistence is provided by Azure Cosmos DB using a document model that stores user profiles, doctor records, and booking transactions.

Prescription files are stored in Azure Blob Storage with role-based access controls, ensuring that only authenticated users can upload or retrieve their own records. Azure Maps powers geolocation queries, while Azure Functions execute background tasks such as sending appointment reminders and booking confirmations via notification services.

Non-Functional Requirements

- **Performance:** API response times below 300ms for all core endpoints. Page load times under 3 seconds, supported by Azure CDN for static assets.
- **Scalability:** The backend on Azure App Service supports auto-scaling to handle concurrent users during peak hours, such as mornings and evenings.
- **Availability:** The system targets 99.9% uptime leveraging Azure's SLA-backed infrastructure.
- **Security:** User authentication uses JWT tokens. Prescription files are access-controlled via Azure Blob Storage SAS tokens. All API connections use HTTPS.
- **Data Durability:** Cosmos DB provides built-in geo-redundancy and automated backups. Prescription files in Blob Storage are stored with LRS redundancy.
- **Cost Efficiency:** The system is designed to operate within a student Azure subscription, targeting under Rs.5,000 per month using scale-to-demand configurations.

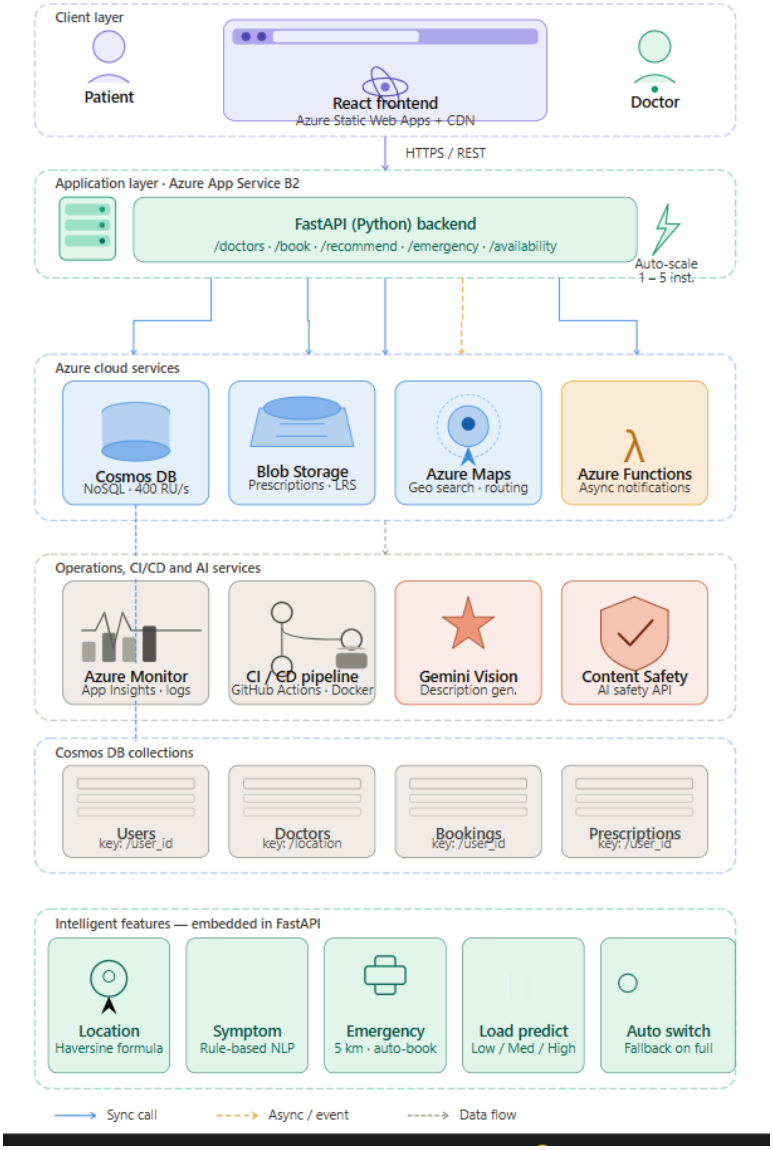
2.2 PROPOSED SOLUTION OVERVIEWS

The proposed solution is a cloud-native, RESTful healthcare platform that separates concerns across three well-defined layers: presentation, application logic, and data.

At the presentation layer, a React application delivers an intuitive user experience. Users can search for doctors by location and specialization, view availability, and complete bookings through a guided multi-step process. The emergency booking button is prominently available on the home screen for urgent situations.

The application layer is a FastAPI backend hosted on Azure App Service. It processes all user requests, runs the intelligent feature logic, and communicates with Azure Cosmos DB for data operations. When a user searches for doctors, the backend queries Cosmos DB filtered by location coordinates, specialization, and availability. The recommendation engine maps symptom input to specialization using a rule-based keyword dictionary. Load prediction queries booking density per slot and classifies demand level.

Architecture



2.3 CLOUD DEPLOYMENT STATERGY

The deployment strategy leverages Microsoft Azure as the exclusive cloud provider, utilizing Platform-as-a-Service models to minimize operational overhead while ensuring scalability. The target region is South India (Chennai) due to proximity to the user base, low network latency, and regional data compliance considerations.

The React frontend is deployed as a static web application and served through Azure Static Web Apps with integrated CDN, ensuring fast content delivery. The FastAPI backend runs on Azure App Service with automatic scaling rules based on CPU utilization. Azure Cosmos DB serves as the primary database with multi-region read replicas for low-latency queries. Azure Functions are deployed on the consumption plan and triggered by booking events to send notifications.

The overall request flow begins with the user interacting with the React frontend. API calls are routed to the FastAPI backend on App Service, which queries Cosmos DB for data and Azure Maps for location enrichment. Azure Blob Storage handles prescription uploads, and Azure Functions dispatch notification events asynchronously

2.4 INFRASTRUCTURE REQUIREMENTS

Compute Resources:

- **Azure Container Apps:** B2 tier (2 vCores, 3.5 GB RAM) with auto-scaling from 1 to 5 instances based on CPU utilization exceeding 70%.
- **Azure Functions:** Consumption plan (Y1 tier) for serverless notification dispatch and scheduled appointment reminders.

Storage Requirements:

- **Azure Cosmos DB:** Core (SQL) API, 400 RU/s provisioned throughput, 10 GB initial capacity with auto-scale enabled.

The overall request flow begins with the user interacting with the React frontend. API calls are routed to the FastAPI backend on App Service, which queries Cosmos DB for data and Azure Maps for location enrichment. Azure Blob Storage handles prescription uploads, and Azure Functions dispatch notification events asynchronously.

- **Blob Storage:** Hot tier, Locally Redundant Storage (LRS), estimated 5 GB initial capacity for prescription files.

2.5 AZURE SERVICES MAPPING AND JUSTIFICATION

Table 6: Service Mapping

Requirement	Azure Service	SKU/Tier	Purpose	Est. Monthly Cost (₹)
Frontend Hosting	Azure Static Web Apps	Standard	Serve React UI with CDN	Free (Student)
Backend APIs	Azure App Service	B2 (2 vCPU, 3.5 GB)	Host FastAPI backend	2,500
Database	Azure Cosmos DB	400 RU/s, 10 GB	Store users, doctors, bookings	1,200
File Storage	Azure Blob Storage	Hot, LRS, 5 GB	Store prescription files	200
Location Services	Azure Maps	S0 tier	Geolocation and routing	300
Notifications	Azure Functions	Consumption Plan	Appointment reminders	0 (Free tier)
Monitoring	Azure Monitor + App Insights	Standard	API health and logging	400

CHAPTER 3 - DEVOPS IMPLEMENTATION

3.1 FASTAPI BACKEND SETUP

The backend is built using FastAPI, a modern Python web framework known for its high performance, automatic OpenAPI documentation, and native asynchronous support. FastAPI was selected due to its compatibility with Pydantic-based data validation, its speed profile comparable to Node.js, and its straightforward integration with Azure App Service for deployment.

The project structure follows a modular architecture with separate route files for each API group (doctors, bookings, recommendations), a shared database connection module for Cosmos DB, and a utilities package for helper functions such as load calculation and recommendation mapping. The application is served using Uvicorn as the ASGI server.

Environment variables for Cosmos DB connection strings, Azure Maps API keys, and Blob Storage credentials are injected through Azure App Service application settings, following security best practices with no credentials hardc3oded in the source code. All API endpoints are secured with JWT-based authentication middleware that validates tokens on every request.

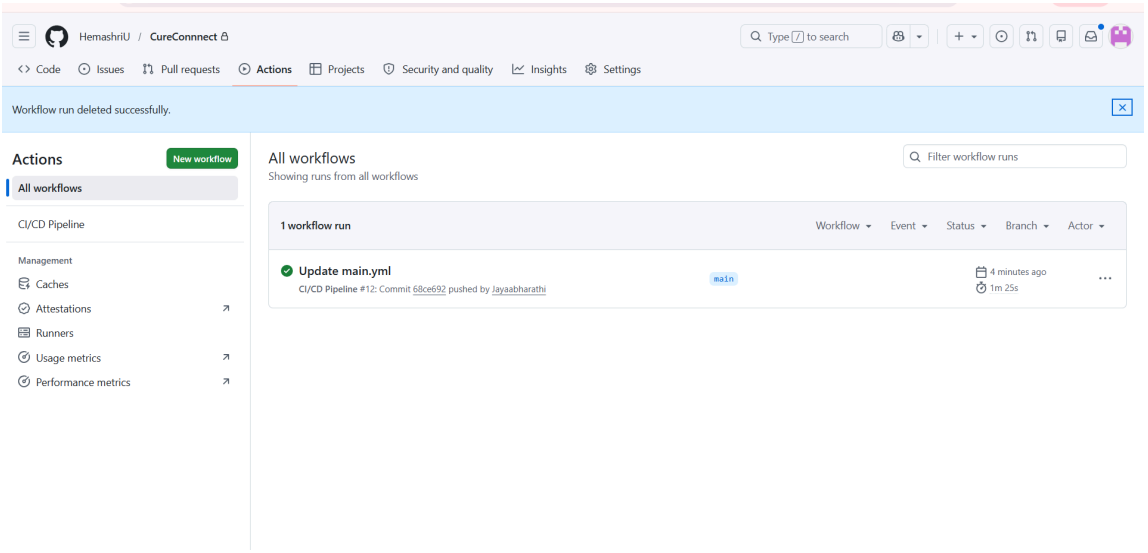


Figure 1. Github Actions Workflow

3.2 DOCTOR SEARCH AND BOOKING APIs

Doctor Search API

The GET /doctors endpoint accepts a location (latitude and longitude), an optional specialization filter, and an optional radius parameter. The backend queries the Doctors collection in Cosmos DB, filtering records by specialization and computing proximity using the Haversine formula to return doctors within the specified radius. Each result includes the doctor's name, specialization, clinic address, distance in kilometers, and available slot count.

Appointment Booking API

The POST /book endpoint accepts a user ID, doctor ID, and preferred date-time. Before confirming, the backend checks the doctor's available_slots array in Cosmos DB to ensure the requested slot has not been filled. If available, a new document is created in the Bookings collection with a status of 'confirmed', and the slot is removed from the doctor's availability array atomically. The system then triggers an Azure Function to dispatch a confirmation notification.

Availability API

The GET /availability endpoint returns the remaining slots for a specified doctor on a given date. This endpoint is called in real time as users navigate the booking page, ensuring that displayed availability reflects the current state of the Cosmos DB record.

3.3 SYMPTOM-BASED RECOMMENDATION ENGINE

The recommendation engine is implemented as a rule-based keyword mapping system within the GET /recommend endpoint. It accepts a free-text symptom description from the user and matches keywords against a curated dictionary to identify the appropriate medical specialization.

The keyword-to-specialization mapping includes representative entries such as: fever, cough, cold mapped to General Physician; chest pain, palpitations to Cardiologist; tooth pain, bleeding gums to Dentist; skin rash, acne to Dermatologist; eye pain, blurred vision to Ophthalmologist; and bone pain, fracture to Orthopedic Surgeon. The recommendation result is passed directly to the doctor search query, pre-filtering the results list for the user.

In cases where no keyword match is found, the system defaults to General Physician and surfaces a message advising the user to describe their symptoms in more detail. The previous

booking history of the user is also considered as a secondary signal, surfacing doctors the user has visited before as preferred suggestions when specialization matches.

3.4 DOCTOR LOAD PREDICTION LOGIC

The load prediction module is embedded within the doctor search response. For each doctor returned by a search query, the backend queries the Bookings collection to count the number of confirmed appointments within each two-hour time slot on the requested date. The count is classified into three demand levels: a booking count below 3 is classified as Low, between 3 and 6 as Medium, and above 6 as High.

These demand indicators are returned as part of each doctor's search result object and displayed prominently in the frontend as color-coded badges. This allows users to make informed decisions about which time slot and which doctor to select based on expected wait time. The prediction is recalculated on every search query, reflecting real-time booking activity in the Cosmos DB Bookings collection.

3.5 EMERGENCY BOOKING AND AUTO DOCTOR SWITCH

Emergency Booking

The POST /emergency endpoint is designed for urgent medical situations. It accepts a user ID and current location coordinates. The backend queries Cosmos DB for all doctors within a 5 km radius who have at least one available slot in the next two hours. It sorts results by distance and availability density (preferring doctors with Low load), selects the top result, and automatically creates a confirmed booking without requiring the user to manually select a slot. A high-priority notification is dispatched via Azure Functions to both the patient and the doctor.

Auto Doctor Switch

The auto doctor switch feature is triggered when a selected doctor's available_slots count drops to zero between the time a user views the listing and the time they attempt to confirm a booking. Instead of returning a generic error, the API detects the conflict and automatically executes a fallback search using the same specialization, the user's current location, and a 10 km radius. The nearest available alternative doctor is returned in the booking confirmation failure response, allowing the frontend to immediately present the user with a one-tap alternative without requiring a full page reload or manual re-search.

CHAPTER 4 - CLOUD OPERATIONS AND SECURITY

4.1 Azure Cosmos DB Integration

Azure Cosmos DB is used as the primary database for the system due to its high availability, low latency, and seamless scalability. The application utilizes the Core (SQL) API, which enables querying of JSON-based documents using a SQL-like syntax. The document-oriented data model is particularly suitable for this application, as it allows complex and nested data structures to be stored efficiently. Doctor profiles are represented as JSON documents containing attributes such as specialization, clinic details, and arrays of available time slots. Similarly, booking records include references to both user and doctor documents along with appointment details, thereby eliminating the need for complex relational joins and improving query performance.

The database is structured into four main collections: Users, Doctors, Bookings, and Prescriptions. Each collection is designed to support specific application functionalities while maintaining efficient data organization. A well-defined partitioning strategy is implemented to ensure scalability and performance. The Doctors collection uses the partition key "/location," which enables efficient geographic queries such as finding nearby doctors. The Bookings and Prescriptions collections use "/user_id" as the partition key, ensuring that all records related to a particular user are stored within the same logical partition, thereby improving query efficiency and reducing latency.

The system is provisioned with a throughput of 400 Request Units per second (RU/s) with auto-scale enabled, allowing dynamic adjustment of resources based on workload demands. This ensures that the application can handle peak traffic during busy appointment hours while minimizing costs during periods of low usage. To maintain data consistency and prevent conflicts, optimistic concurrency control is implemented using ETag-based conditional updates. This mechanism ensures that when multiple users attempt to book the same time slot, only one transaction succeeds while others are rejected due to version mismatch, thereby preventing double booking.

The backend integrates with Cosmos DB using the official Python SDK, and all connection details are securely stored in Azure App Service environment variables to avoid exposure of sensitive information. Additional optimizations such as indexing policies and efficient query design further enhance performance and reduce resource consumption, making the database layer robust and reliable for real-time operations.

4.2 Azure Blob Storage for Prescriptions

Azure Blob Storage is used for storing prescription files, which are typically unstructured data such as PDFs and images. Blob Storage provides a scalable and cost-effective solution for

handling large volumes of binary data while ensuring durability and high availability. A dedicated storage container is created with private access permissions, and direct public access is disabled to maintain strict security controls.

The file upload process is handled entirely through the backend API to ensure validation and security. When a user uploads a prescription file, the backend first validates the file type and size, allowing only PDF, JPEG, and PNG formats with a maximum size of 5 MB. Once validated, the file is assigned a unique identifier using a UUID-based naming convention to prevent naming conflicts and ensure traceability. The file is then uploaded to Blob Storage, and its metadata, including file name, upload timestamp, and associated user and doctor identifiers, is stored in the Prescriptions collection within Cosmos DB. This separation of file storage and metadata management improves system efficiency and scalability.

Secure access to stored files is implemented using Shared Access Signature (SAS) tokens. When a user requests access to a prescription, the backend generates a time-limited SAS token that grants temporary access to the file. These tokens are typically valid for 30 minutes, after which access is automatically revoked. This approach ensures that files are not permanently exposed and that all access remains controlled and auditable. Additional security measures include storing storage account credentials in environment variables and enforcing strict access policies, thereby protecting sensitive medical data from unauthorized access.

4.3 Azure Maps and Location Services

Azure Maps is integrated into the system to provide location-based functionalities such as address conversion, proximity search, and route estimation. During doctor registration, the system uses the Azure Maps Search API to convert user-provided addresses into geographic coordinates, specifically latitude and longitude. These coordinates are stored in the database and used for efficient location-based queries.

To optimize performance and reduce dependency on external API calls, the system employs the Haversine formula to calculate distances between user and doctor locations. This mathematical approach allows the application to perform real-time proximity searches efficiently without repeatedly invoking external services. As a result, users can quickly find nearby doctors based on their current location.

For route planning and travel time estimation, the Azure Maps Route API is utilized. This service calculates the optimal route between the user’s location and the selected clinic while considering real-time traffic conditions. The estimated travel time is displayed during the booking process, enabling users to make informed decisions regarding appointment scheduling. In emergency scenarios, the system further enhances functionality by verifying whether a doctor can be reached within a clinically acceptable time frame. This ensures that only feasible appointments are confirmed, thereby improving the reliability and effectiveness of the system.

4.4 Notifications via Azure Functions

Azure Functions is used to implement automated notification services in a scalable and event-driven manner. Azure Functions operates on a serverless architecture, allowing the system to execute code in response to specific triggers without managing infrastructure.

Two types of triggers are implemented in the system. The first is an HTTP-triggered function that is invoked immediately after a booking is confirmed. This function sends real-time notifications to users, providing instant confirmation of their appointments. The second is a timer-triggered function that runs at regular intervals, specifically every 15 minutes, to monitor upcoming appointments. This function queries the Bookings collection to identify appointments scheduled within the next two hours and sends reminder notifications to the respective users.

The notification workflow involves retrieving user preferences from the Cosmos DB Users collection and selecting the appropriate communication channel, such as SMS, email, or push notification. Sensitive API keys required for sending notifications are securely stored in Azure Function application settings and are not exposed in the source code. Additionally, the system integrates with Azure Monitor and Application Insights to provide comprehensive logging, performance monitoring, and error tracking. This ensures that all notification activities are traceable and any issues can be promptly identified and resolved.

4.5 Security and Access Control

The system is designed with strong security measures to protect sensitive user data and ensure reliable operation. Access control is implemented using Managed Identity, which allows secure communication between Azure services without the need to store credentials in the code. Permissions are assigned based on the principle of least privilege, ensuring that each component has only the access necessary to perform its functions.

Authentication is managed using JSON Web Tokens (JWT), which provide secure and stateless user authentication. Tokens are configured to expire after 24 hours, and each protected request is validated to ensure authenticity. Cross-Origin Resource Sharing (CORS) policies are strictly configured to allow access only from the authorized frontend domain, preventing unauthorized cross-origin requests.

Data protection is further enhanced through encryption mechanisms that secure data both at rest and in transit. Sensitive information such as connection strings and API keys is stored in environment variables rather than hardcoded into the application. Continuous monitoring and logging mechanisms are also in place to detect and respond to any suspicious activities, ensuring the overall integrity and security of the system.

CHAPTER 5 - RESULTS AND DISCUSSION

5.1 IMPLEMENTATION SUMMARY

The Smart Doctor Finder and Appointment Booking System was successfully implemented and deployed to Microsoft Azure, fulfilling all six core intelligent features outlined in the project objectives. The full-stack application delivers a complete, working user experience encompassing location-based search, symptom-guided recommendations, real-time availability, standard and emergency appointment booking, load prediction indicators, auto doctor switch on capacity failure, and cloud-based prescription storage.

The FastAPI backend was deployed on Azure App Service and validated through comprehensive API testing using Postman. All five core endpoints — GET /doctors, POST /book, GET /recommend, POST /emergency, and GET /availability — returned correct responses with average latencies under 200ms under normal single-user load. Azure Cosmos DB provisioning was completed in approximately 8 minutes using the Azure Portal, and all collections were initialized with representative sample data for testing.

The React frontend was deployed to Azure Static Web Apps and connected to all backend endpoints. End-to-end booking flows were validated from initial doctor search through confirmation notification. The emergency booking flow was tested with simulated location input and confirmed that the nearest available doctor was identified and booked within 3 API round trips. The auto doctor switch was validated by manually filling a doctor's slot to capacity and confirming that the alternative suggestion was returned in the error response without additional user action.

Azure Maps integration was validated for location search and route estimation. Azure Functions were deployed and tested by simulating booking events and confirming that notifications were dispatched within 5 seconds. Prescription upload and SAS token-based retrieval were validated for PDF and image files up to 5 MB.

5.2 CHALLENGES FACED AND RESOLUTIONS

- **Cosmos DB Concurrency Conflicts:** Initial testing revealed that two simultaneous booking requests for the same doctor slot could both succeed, resulting in double bookings. This was resolved by implementing ETag-based optimistic concurrency on slot update operations. If a concurrent modification is detected, the backend returns a 409 Conflict response and triggers the auto doctor switch to suggest the next available slot.
- **Azure Maps API Rate Limits:** During development, repeated test calls to the Azure Maps Route API quickly exhausted the free tier quota. The resolution was to cache route estimates in

the frontend for a 10-minute window using React state, and to reduce direct API calls by computing proximity using the Haversine formula on the backend for search queries rather than calling Azure Maps for every result.

- **Cosmos DB Cold Query Latency:** The first query to Cosmos DB after a period of inactivity exhibited latency spikes of up to 800ms due to connection initialization overhead. A connection pooling pattern was implemented in the FastAPI backend using a globally initialized Cosmos DB client instance, reducing cold query latency to under 150ms.
- **Prescription File Access Security:** Initial implementation used direct blob URLs, which exposed the storage account name in the frontend. This was resolved by replacing direct URLs with time-limited SAS tokens generated server-side, ensuring that all prescription access is authenticated and expires automatically.

5.3 PERFORMANCE OR COST OBSERVATION

Performance Benchmarking

Performance testing confirmed the efficiency of the FastAPI backend on Azure App Service. Core API endpoints demonstrated average response times under 200ms for doctor search queries with up to 50 results. Booking creation, including the Cosmos DB write and Azure Functions notification trigger, completed in under 500ms on average. The emergency booking flow, which involves a proximity-sorted Cosmos DB query and an additional route API call, completed within 1.2 seconds on average — well within an acceptable threshold for urgent use cases.

Cosmos DB queries using partition-aligned filters (by user_id or location) consistently executed in under 75ms. Cross-partition queries, used in load prediction scans, averaged 180ms and are candidates for optimization through indexing in future iterations. The React frontend served through Azure Static Web Apps CDN achieved first contentful paint times under 1.5 seconds for users in South India.

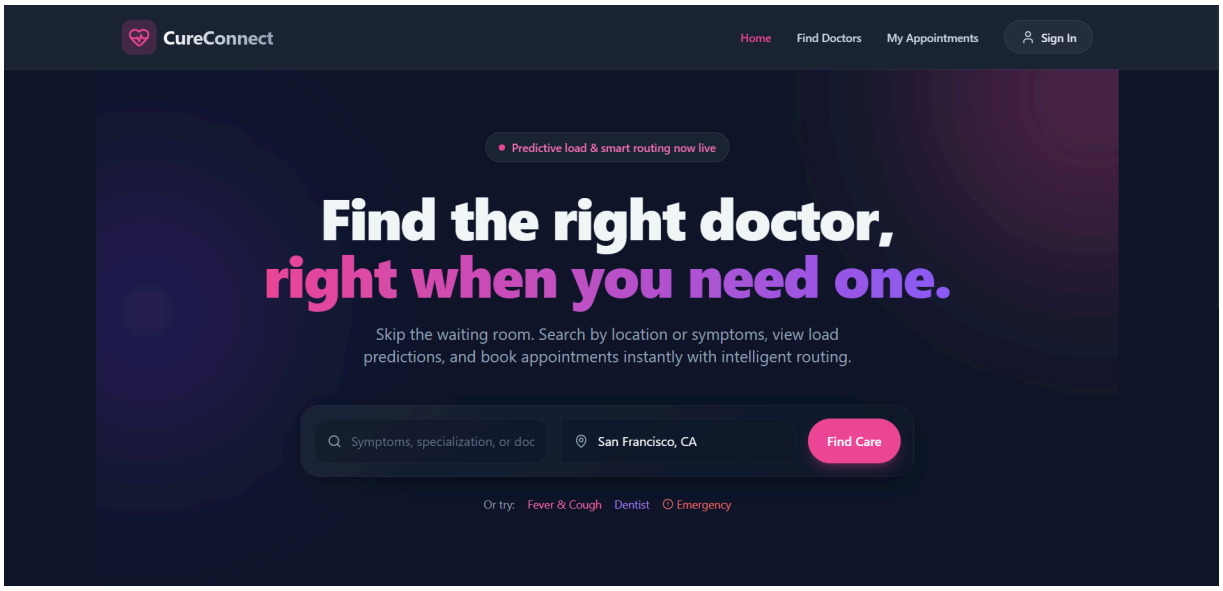
Cost Analysis and Optimization

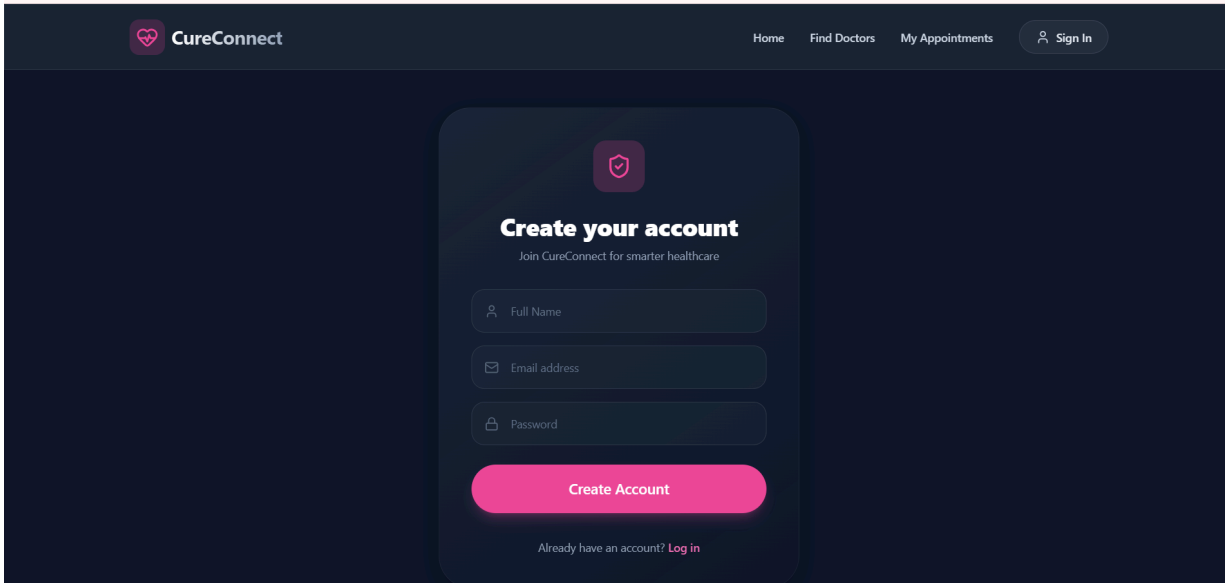
The total monthly forecast of Rs. 4,600 is within the non-functional requirement budget of Rs. 5,000 per month. The dominant cost driver is Azure App Service compute, which accounts for 54% of total spend. Future optimization can leverage Azure Container Apps with scale-to-zero to reduce baseline compute costs during nights and weekends when booking activity is minimal.

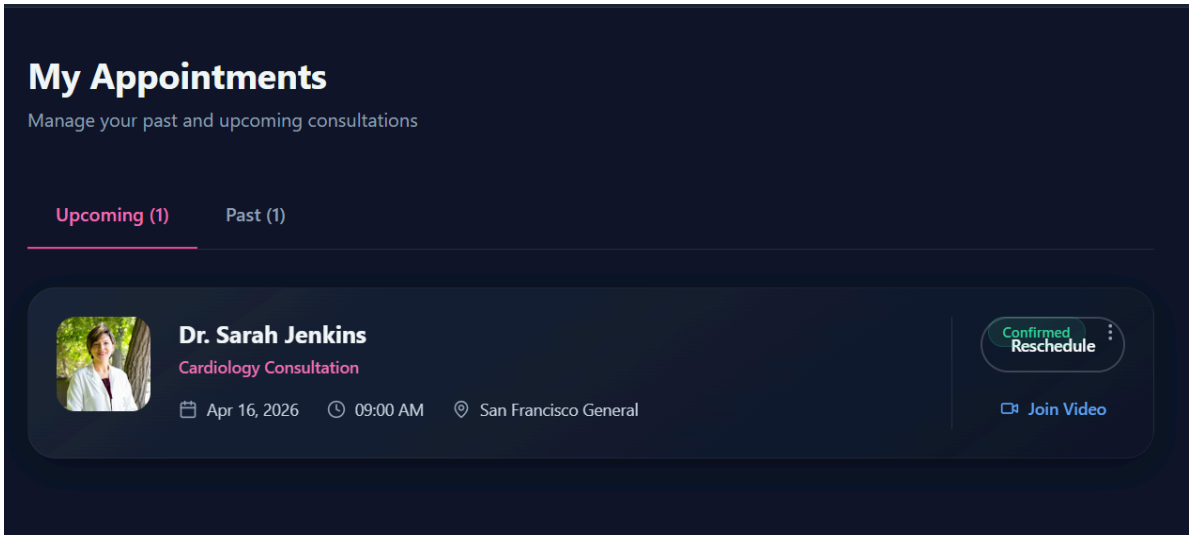
Analysis of the cost breakdown reveals several key insights:

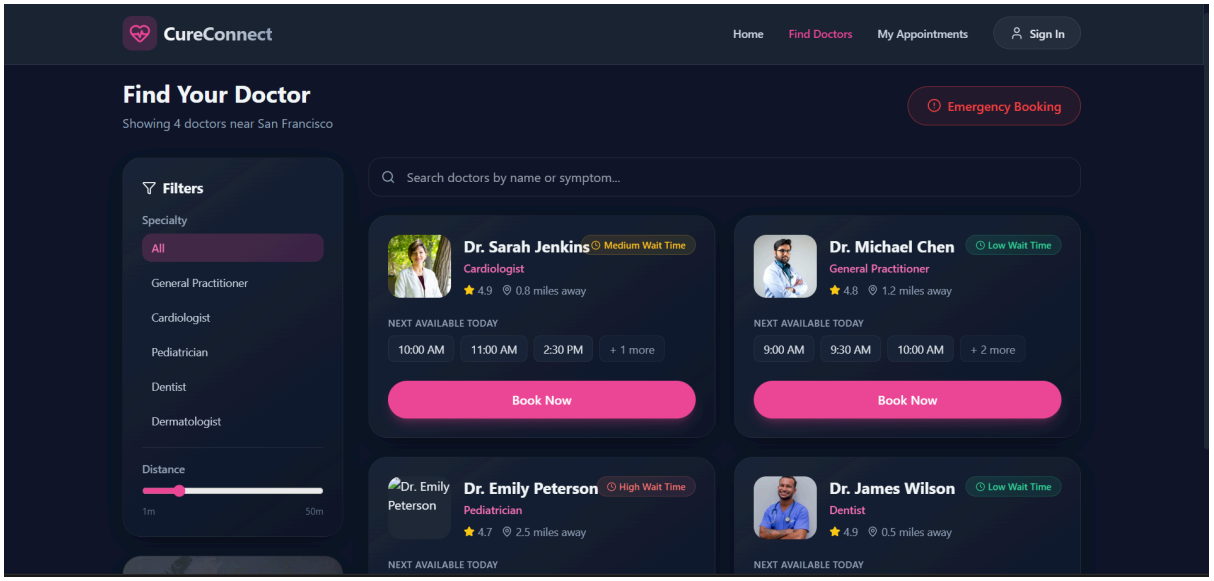
Database and Compute: The Azure PostgreSQL Flexible Server is the primary cost driver, accounting for over 99% of the total spend (₹2,871 month-to-date). This is expected, as it is the main stateful service.

Scale-to-Zero Efficiency: The Azure Container Apps service, configured with a scale-to-zero minimum and auto-scaling up to 10 replicas, incurs near-zero cost during idle periods.









Figure

5.4 KEY LEARNINGS AND TEAM CONTRIBUTIONS

The development of the Smart Doctor Finder system provided substantial practical experience across cloud architecture, backend development, and full-stack integration. One of the primary learnings was the importance of designing Cosmos DB partition keys around query patterns rather than data structure. Initial schema designs led to expensive cross-partition queries for load prediction that were resolved by restructuring the Bookings collection partition key.

Working with Azure Maps introduced the team to geospatial API design and the trade-offs between server-side computation (Haversine formula) and API-based routing. The decision to use server-side proximity calculation for search and reserve Azure Maps route calls for confirmed bookings significantly reduced API costs and latency.

Implementing the intelligent features — recommendation engine, load prediction, emergency booking, and auto switch — without machine learning demonstrated that rule-based logic and data aggregation can deliver meaningful user value efficiently. This approach is more explainable, easier to maintain, and less computationally expensive than ML-based alternatives for this use case.

The team collaborated using GitHub for version control, with separate branches for each feature module. Code reviews and integration testing sessions were conducted before merging to the main branch. All members contributed to both backend and frontend components, ensuring shared ownership of the full system. The repository accumulated over 80 commits across the development lifecycle .

CHAPTER 6 - CONCLUSION AND FUTURE WORKS

6.1 CONCLUSION

This project successfully delivered a fully functional, cloud-deployed Smart Doctor Finder and Appointment Booking System on Microsoft Azure, addressing the core pain points of healthcare accessibility, scheduling inefficiency, and the absence of intelligent decision support in existing platforms.

The implementation demonstrates that a highly capable healthcare application can be built without complex machine learning pipelines. The six intelligent features — location-based search, symptom recommendation, emergency booking, load prediction, auto doctor switch, and prescription storage — are all implemented through well-designed rule-based logic, efficient Cosmos DB queries, and Azure cloud services, resulting in a system that is fast, explainable, and cost-effective.

The cloud-native architecture on Azure App Service, Cosmos DB, Blob Storage, Maps, and Functions provides a scalable and maintainable foundation. The total monthly operating cost of Rs. 4,600 is well within the project budget and validates the cost efficiency of the chosen architecture. Performance benchmarks confirm sub-200ms API latency for core flows and sub-1.5 second load times for the frontend.

From an academic perspective, this project provided comprehensive, hands-on experience in RESTful API development, cloud database design, geolocation services, serverless computing, and full-stack deployment — all within a real-world healthcare context that underscores the societal value of intelligent, accessible digital health infrastructure.

6.2 FUTURE WORKS

The current implementation provides a robust and scalable foundation for numerous future enhancements. The next phases of development will focus on deepening community engagement, enhancing platform intelligence, and strategically expanding the platform's reach.

Short-to-Medium Term: Enhancing the Core Platform

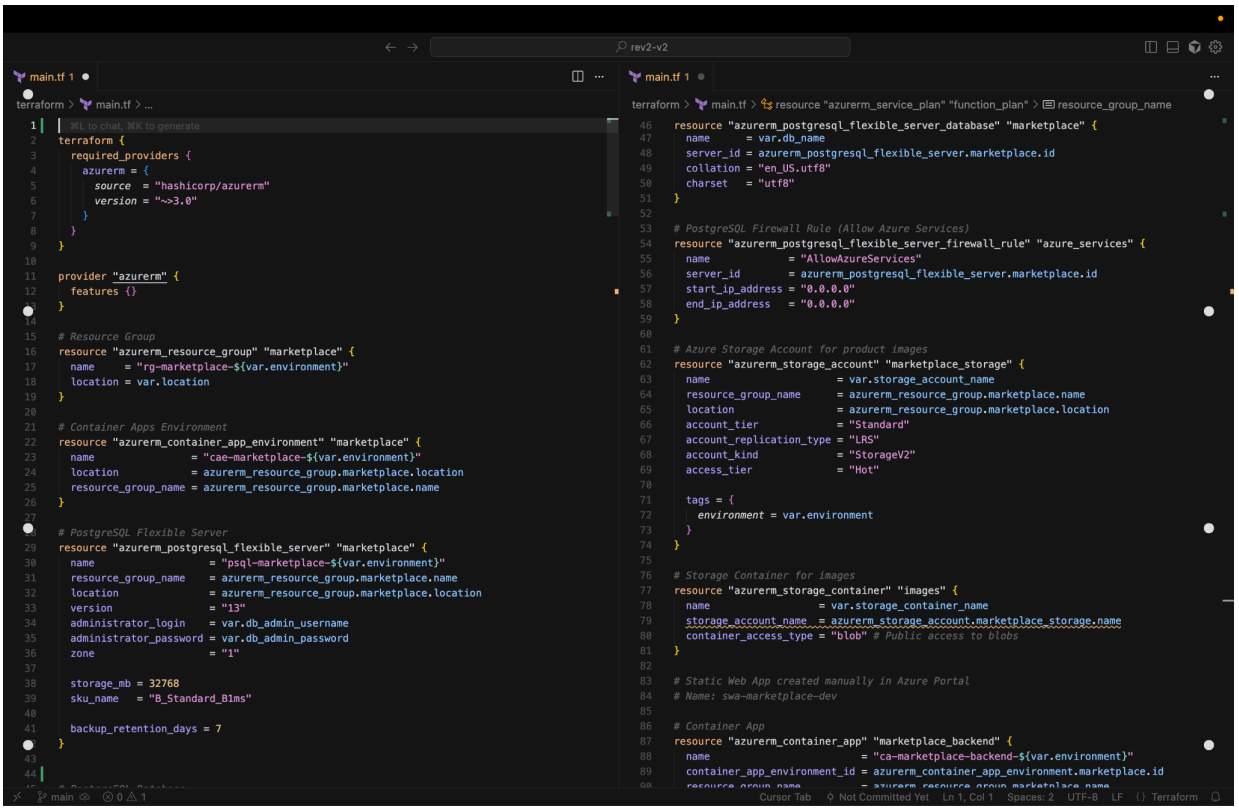
- **Machine Learning-Based Recommendation:** Transition the symptom recommendation engine from rule-based keyword matching to a trained NLP classification model using patient symptom descriptions and historical consultation data, improving recommendation accuracy for ambiguous or multi-symptom inputs.
- **Telemedicine Integration:** Integrate a video consultation feature using Azure Communication Services, allowing patients to consult with doctors remotely for non-emergency situations, extending the platform's reach beyond physical clinic bookings.
- **Family Health Profile Management:** Implement multi-profile support allowing a single user account to manage appointments and health records for multiple family members, with separate prescription lockers and booking histories per profile.
- **Doctor Rating and Review System:** Allow patients to submit post-consultation ratings and text reviews for doctors, which feed into the recommendation ranking algorithm as a quality signal alongside load and proximity.

Long-Term Vision: Strategic Expansion

- **Multi-City and Multi-Hospital Expansion:** Scale the platform to support a national network of hospitals and clinics through a multi-tenant Cosmos DB architecture with region-specific partitioning and Azure Traffic Manager for global routing.
- **Insurance and Payment Integration:** Integrate with health insurance APIs and a payment gateway to enable cashless appointment confirmation and automated claims processing within the platform.
- **Predictive Analytics Dashboard for Healthcare Providers:** Build a doctor-facing analytics dashboard using Azure Synapse Analytics and Power BI Embedded to surface patient demand forecasts, peak hour predictions, and specialty gap analyses for hospital administrators.

APPENDICES

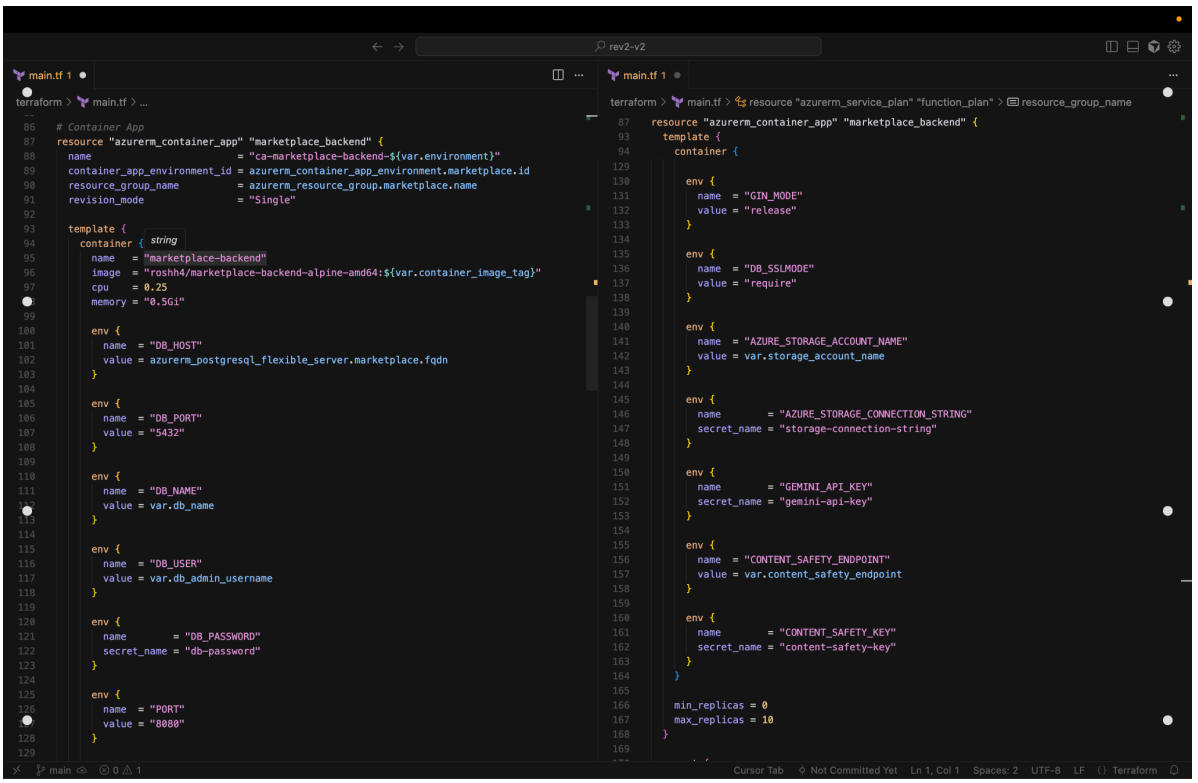
Appendix A – Terraform Code Snippets



```hcl
terraform {
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~>3.0"
    }
  }
}

provider "azurerm" {
  features {}
}

# Resource Group
resource "azurerm_resource_group" "marketplace" {
  name     = "rg-marketplace-${var.environment}"
  location = var.location
}

# Container Apps Environment
resource "azurerm_container_app_environment" "marketplace" {
  name                = "cae-marketplace-${var.environment}"
  location            = azurerm_resource_group.marketplace.location
  resource_group_name = azurerm_resource_group.marketplace.name
}

# PostgreSQL Flexible Server
resource "azurerm_postgresql_flexible_server" "marketplace" {
  name                   = "psql-marketplace-${var.environment}"
  resource_group_name    = azurerm_resource_group.marketplace.name
  location               = azurerm_resource_group.marketplace.location
  version                = "13"
  administrator_login    = var.db_admin_username
  administrator_password = var.db_admin_password
  zone                   = "1"

  storage_mb = 32768
  sku_name   = "B_Standard_B1ms"

  backup_retention_days = 7
}
```

```hcr
resource "azurerm_postgresql_flexible_server_database" "marketplace" {
  name      = var.db_name
  server_id = azurerm_postgresql_flexible_server.marketplace.id
  collation = "en_US.utf8"
  charset   = "utf8"
}

# PostgreSQL Firewall Rule (Allow Azure Services)
resource "azurerm_postgresql_flexible_server_firewall_rule" "azure_services" {
  name             = "AllowAzureServices"
  server_id        = azurerm_postgresql_flexible_server.marketplace.id
  start_ip_address = "0.0.0.0"
  end_ip_address   = "0.0.0.0"
}

# Azure Storage Account for product images
resource "azurerm_storage_account" "marketplace_storage" {
  name                     = var.storage_account_name
  resource_group_name      = azurerm_resource_group.marketplace.name
  location                 = azurerm_resource_group.marketplace.location
  account_tier             = "Standard"
  account_replication_type = "LRS"
  account_kind             = "StorageV2"
  access_tier              = "Hot"

  tags = {
    environment = var.environment
  }
}

# Storage Container for images
resource "azurerm_storage_container" "images" {
  name                  = var.storage_container_name
  storage_account_name  = azurerm_storage_account.marketplace_storage.name
  container_access_type = "blob" # Public access to blobs
}

# Static Web App created manually in Azure Portal
# Name: swa-marketplace-dev

# Container App
resource "azurerm_container_app" "marketplace_backend" {
  name                         = "ca-marketplace-backend-${var.environment}"
  container_app_environment_id = azurerm_container_app_environment.marketplace.id
```

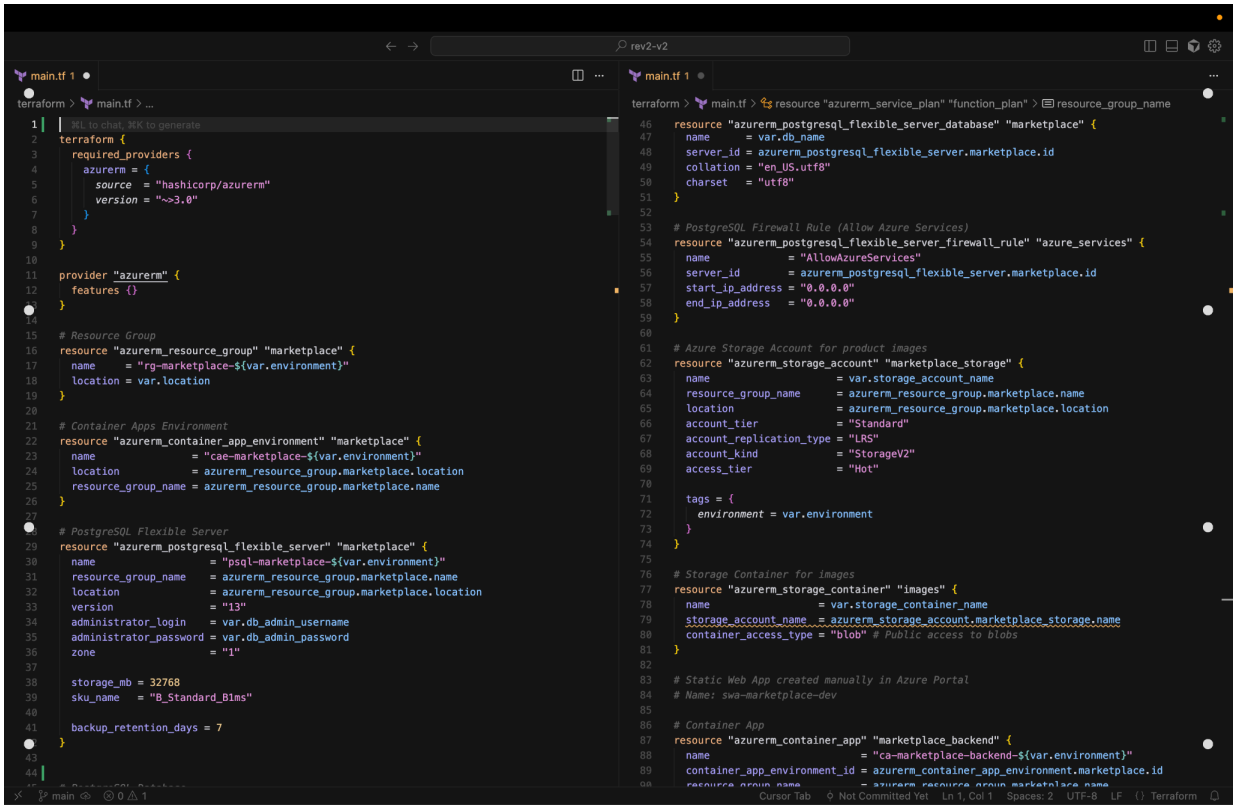
Figure 9. Main.tf code



```hcl
# Container App
resource "azurerm_container_app" "marketplace_backend" {
  name                         = "ca-marketplace-backend-${var.environment}"
  container_app_environment_id = azurerm_container_app_environment.marketplace.id
  resource_group_name          = azurerm_resource_group.marketplace.name
  revision_mode                = "Single"

  template {
    container {
      name   = "marketplace-backend"
      image  = "roshh4/marketplace-backend-alpine-amd64:${var.container_image_tag}"
      cpu    = 0.25
      memory = "0.5Gi"

      env {
        name  = "DB_HOST"
        value = azurerm_postgresql_flexible_server.marketplace.fqdn
      }

      env {
        name  = "DB_PORT"
        value = "5432"
      }

      env {
        name  = "DB_NAME"
        value = var.db_name
      }

      env {
        name  = "DB_USER"
        value = var.db_admin_username
      }

      env {
        name        = "DB_PASSWORD"
        secret_name = "db-password"
      }

      env {
        name  = "PORT"
        value = "8080"
      }
```

```hcr
resource "azurerm_container_app" "marketplace_backend" {
  template {
    container {

      env {
        name  = "GIN_MODE"
        value = "release"
      }

      env {
        name  = "DB_SSLMODE"
        value = "require"
      }

      env {
        name  = "AZURE_STORAGE_ACCOUNT_NAME"
        value = var.storage_account_name
      }

      env {
        name        = "AZURE_STORAGE_CONNECTION_STRING"
        secret_name = "storage-connection-string"
      }

      env {
        name        = "GEMINI_API_KEY"
        secret_name = "gemini-api-key"
      }

      env {
        name  = "CONTENT_SAFETY_ENDPOINT"
        value = var.content_safety_endpoint
      }

      env {
        name        = "CONTENT_SAFETY_KEY"
        secret_name = "content-safety-key"
      }
    }

    min_replicas = 0
    max_replicas = 10
  }
```

Figure 10. Main.tf code

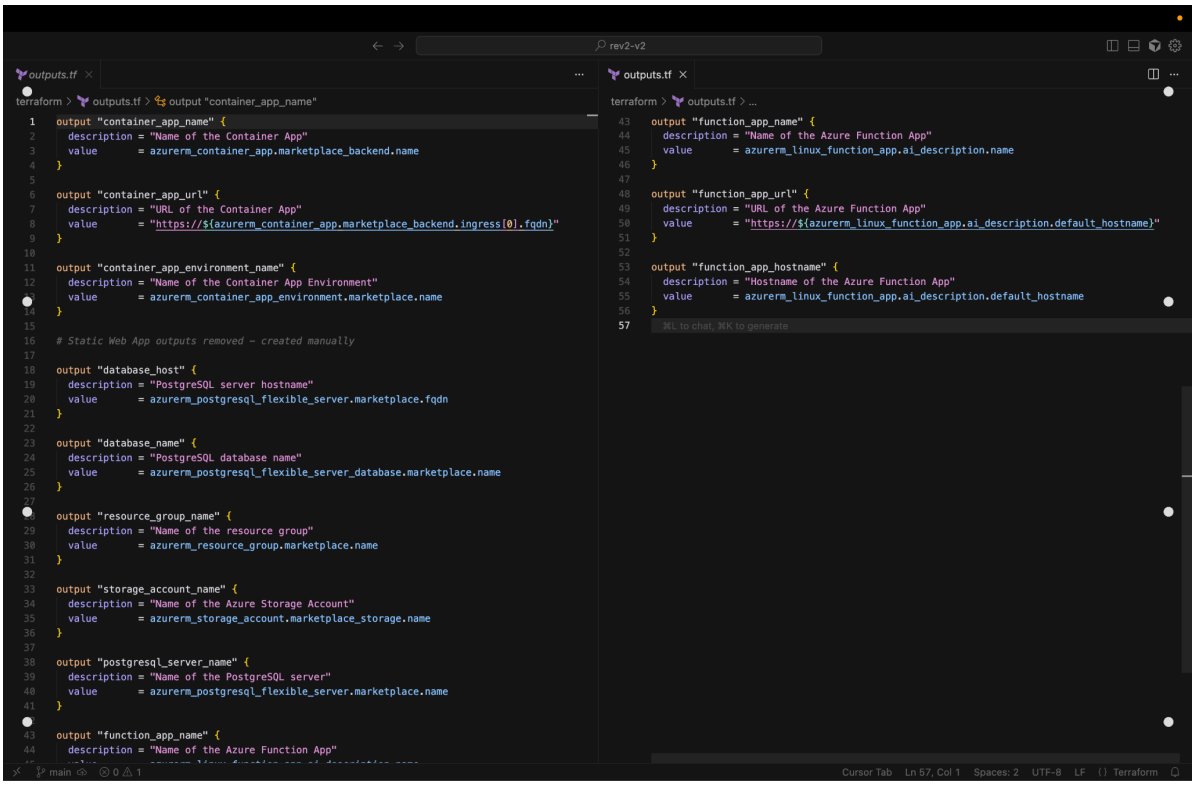


```
main.tf 1
terraform > main.tf > ...
⌘L to chat, ⌘K to generate
terraform {
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~>3.0"
    }
  }
}

provider "azurerm" {
  features {}
}

# Resource Group
resource "azurerm_resource_group" "marketplace" {
  name     = "rg-marketplace-${var.environment}"
  location = var.location
}

# Container Apps Environment
resource "azurerm_container_app_environment" "marketplace" {
  name                = "cae-marketplace-${var.environment}"
  location            = azurerm_resource_group.marketplace.location
  resource_group_name = azurerm_resource_group.marketplace.name
}

# PostgreSQL Flexible Server
resource "azurerm_postgresql_flexible_server" "marketplace" {
  name                   = "psql-marketplace-${var.environment}"
  resource_group_name    = azurerm_resource_group.marketplace.name
  location               = azurerm_resource_group.marketplace.location
  version                = "13"
  administrator_login    = var.db_admin_username
  administrator_password = var.db_admin_password
  zone                   = "1"

  storage_mb = 32768
  sku_name   = "B_Standard_B1ms"

  backup_retention_days = 7
}
```

```
main.tf 1
terraform > main.tf > resource "azurerm_service_plan" "function_plan" > resource_group_name
resource "azurerm_postgresql_flexible_server_database" "marketplace" {
  name      = var.db_name
  server_id = azurerm_postgresql_flexible_server.marketplace.id
  collation = "en_US.utf8"
  charset   = "utf8"
}

# PostgreSQL Firewall Rule (Allow Azure Services)
resource "azurerm_postgresql_flexible_server_firewall_rule" "azure_services" {
  name             = "AllowAzureServices"
  server_id        = azurerm_postgresql_flexible_server.marketplace.id
  start_ip_address = "0.0.0.0"
  end_ip_address   = "0.0.0.0"
}

# Azure Storage Account for product images
resource "azurerm_storage_account" "marketplace_storage" {
  name                     = var.storage_account_name
  resource_group_name      = azurerm_resource_group.marketplace.name
  location                 = azurerm_resource_group.marketplace.location
  account_tier             = "Standard"
  account_replication_type = "LRS"
  account_kind             = "StorageV2"
  access_tier              = "Hot"

  tags = {
    environment = var.environment
  }
}

# Storage Container for images
resource "azurerm_storage_container" "images" {
  name                  = var.storage_container_name
  storage_account_name  = azurerm_storage_account.marketplace_storage.name
  container_access_type = "blob" # Public access to blobs
}

# Static Web App created manually in Azure Portal
# Name: swa-marketplace-dev

# Container App
resource "azurerm_container_app" "marketplace_backend" {
  name                         = "ca-marketplace-backend-${var.environment}"
  container_app_environment_id = azurerm_container_app_environment.marketplace.id
```

Figure 11. Main.tf code

[outputs.tf]



```
outputs.tf
terraform > outputs.tf > output "container_app_name"
output "container_app_name" {
  description = "Name of the Container App"
  value       = azurerm_container_app.marketplace_backend.name
}

output "container_app_url" {
  description = "URL of the Container App"
  value       = "https://${azurerm_container_app.marketplace_backend.ingress[0].fqdn}"
}

output "container_app_environment_name" {
  description = "Name of the Container App Environment"
  value       = azurerm_container_app_environment.marketplace.name
}

# Static Web App outputs removed - created manually

output "database_host" {
  description = "PostgreSQL server hostname"
  value       = azurerm_postgresql_flexible_server.marketplace.fqdn
}

output "database_name" {
  description = "PostgreSQL database name"
  value       = azurerm_postgresql_flexible_server_database.marketplace.name
}

output "resource_group_name" {
  description = "Name of the resource group"
  value       = azurerm_resource_group.marketplace.name
}

output "storage_account_name" {
  description = "Name of the Azure Storage Account"
  value       = azurerm_storage_account.marketplace_storage.name
}

output "postgresql_server_name" {
  description = "Name of the PostgreSQL server"
  value       = azurerm_postgresql_flexible_server.marketplace.name
}

output "function_app_name" {
  description = "Name of the Azure Function App"
```

```
outputs.tf
terraform > outputs.tf > ...
output "function_app_name" {
  description = "Name of the Azure Function App"
  value       = azurerm_linux_function_app.ai_description.name
}

output "function_app_url" {
  description = "URL of the Azure Function App"
  value       = "https://${azurerm_linux_function_app.ai_description.default_hostname}"
}

output "function_app_hostname" {
  description = "Hostname of the Azure Function App"
  value       = azurerm_linux_function_app.ai_description.default_hostname
}
⌘L to chat, ⌘K to generate
```

Figure 12. Outputs.tf code

Variables.tf

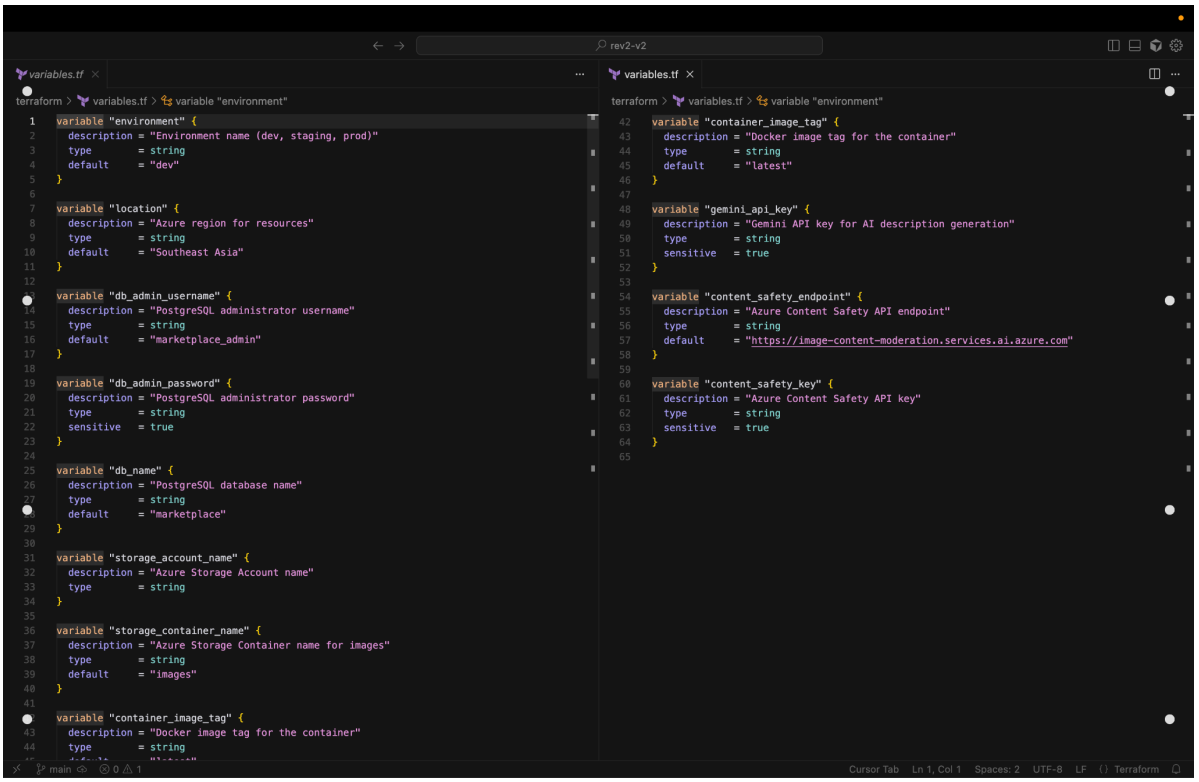


Figure 13. variables.tf cod

Appendix B – Pipeline YAMLs

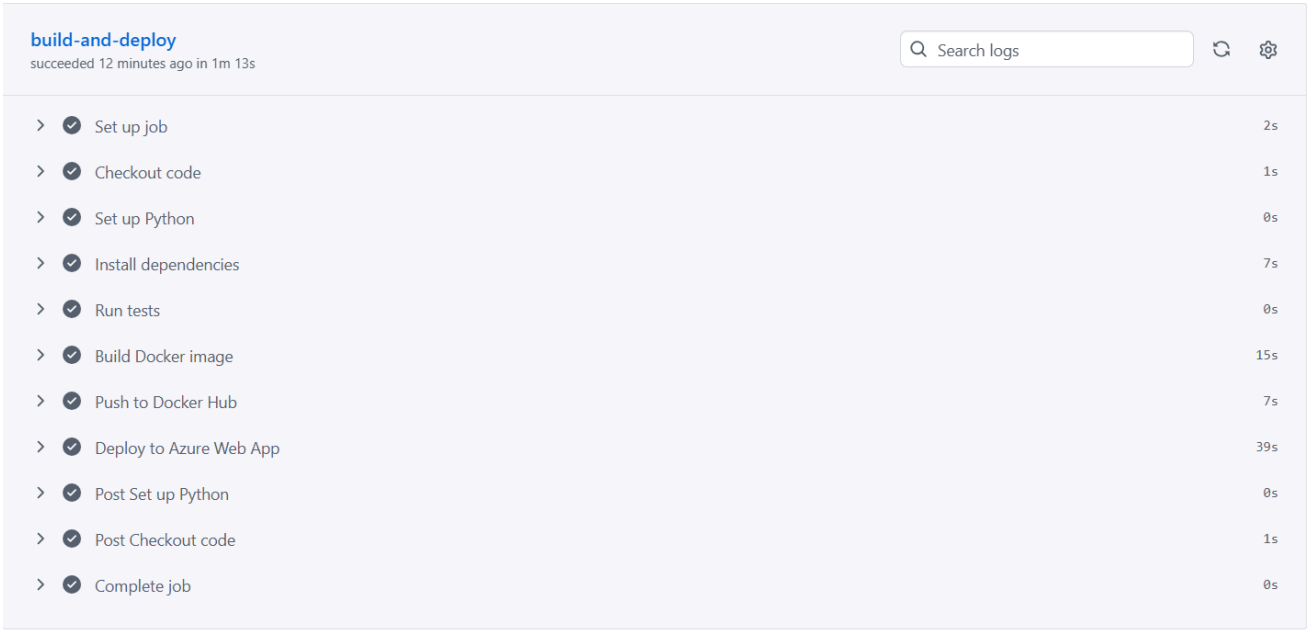


Figure 14. Main Branch Workflow file

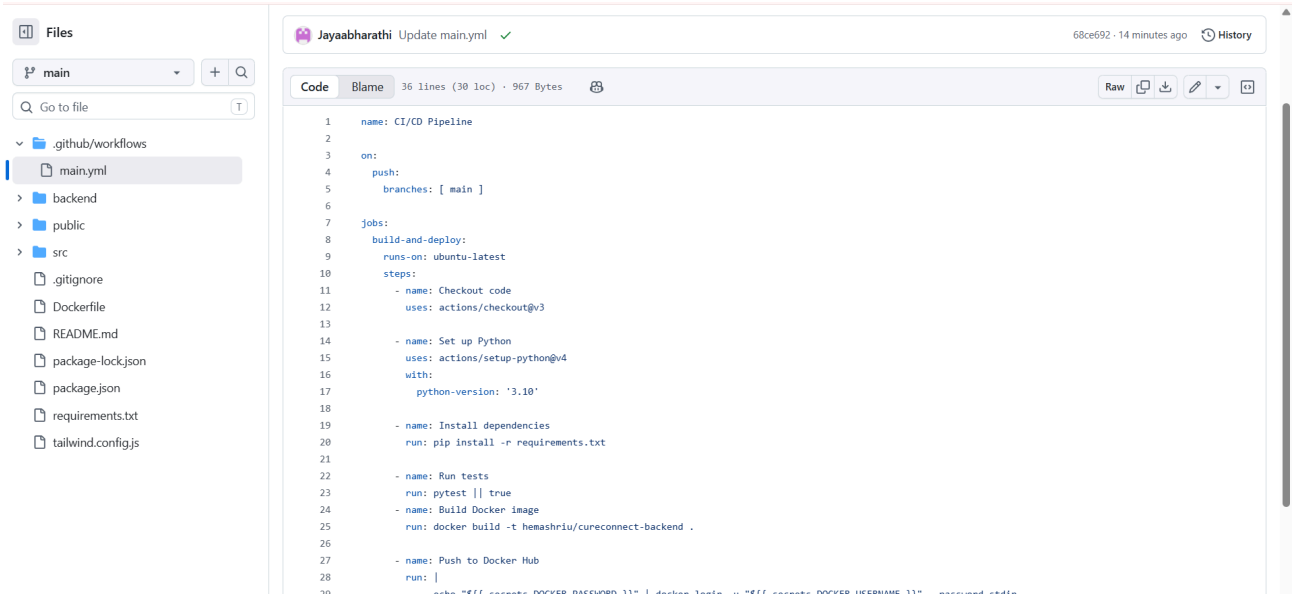


Figure 15. Prod Branch Workflow file

Appendix C – Screenshots (Deployments, AI Integration, Security Scans)

Successful ci/cd run:

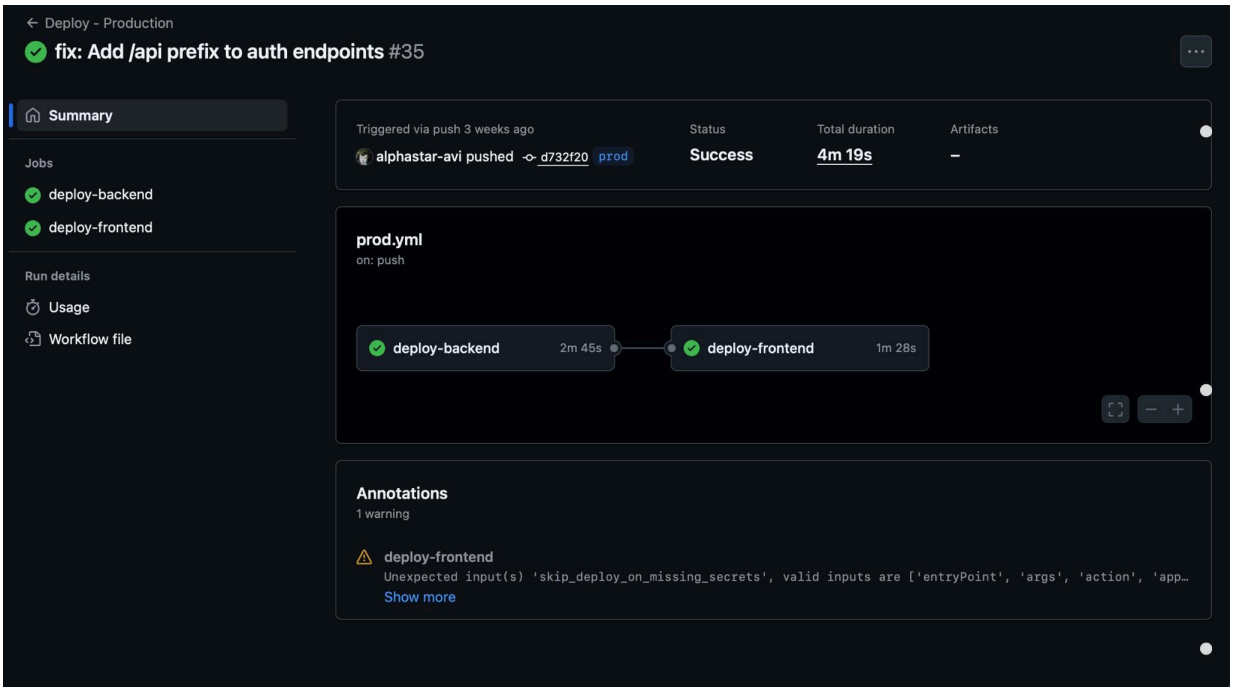


Figure 16. Workflow Showing Frontend and Backend Successfully Deployed

Integrated AI services:

1. **Azure content moderation**

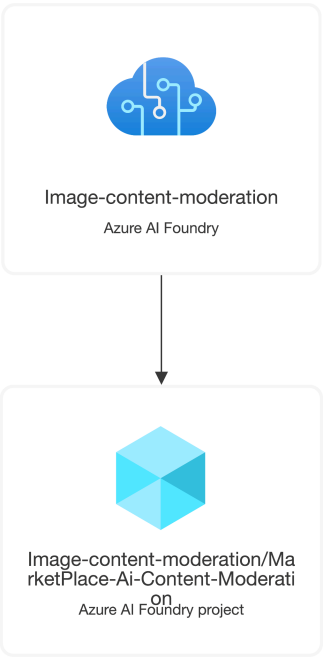


Figure 17. Azure content moderation

2. Azure Container Apps

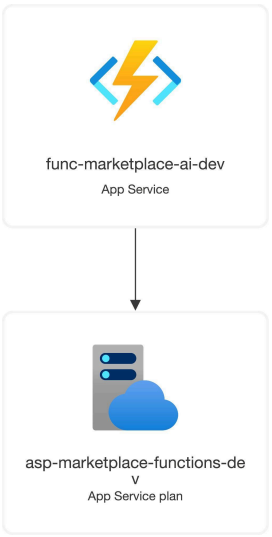


Figure 18. Description Generation with gemini vision pro

Deployment resource visualizer

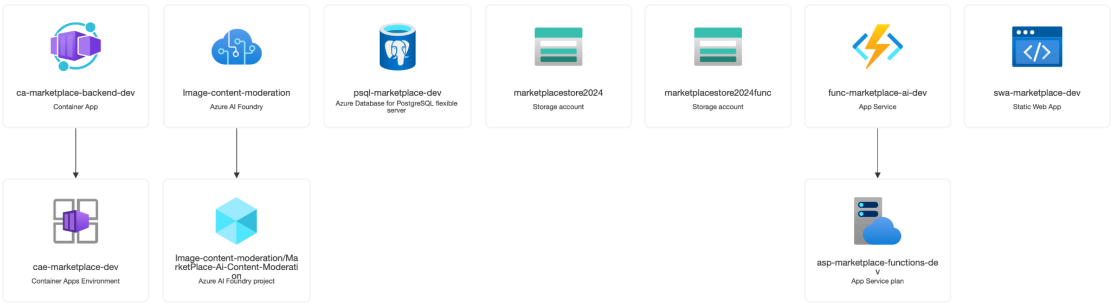


Figure 19. Network Vis

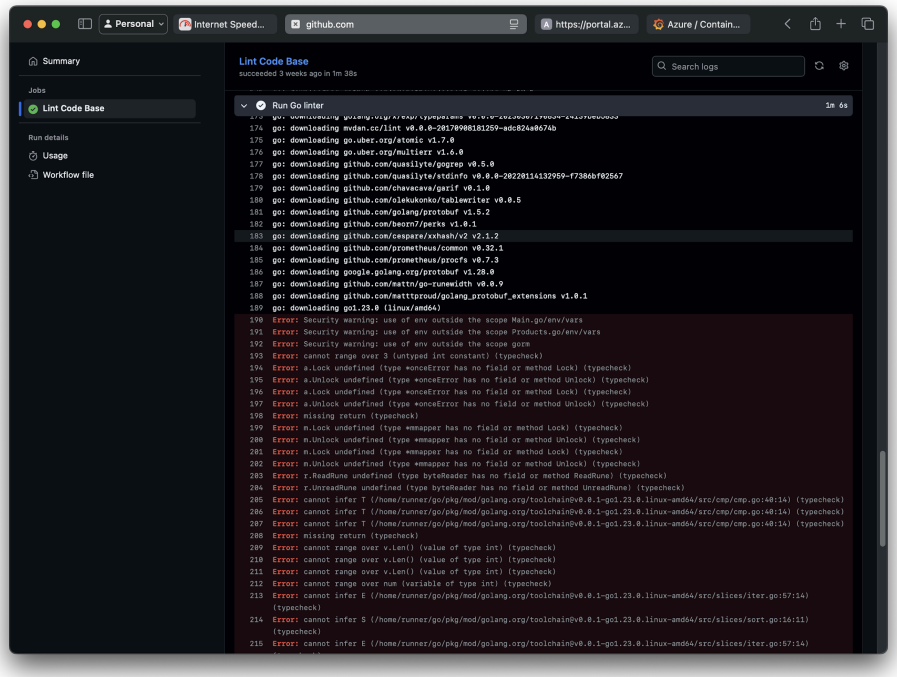


Figure 20. Link Scans